

Paso 2 - Imprimir | Solucionario

Las cinco soluciones del paso 2. Todas son programas sin teclado: la salida es exacta y la tuya tiene que coincidir carácter por carácter, espacios incluidos.

Úsalo después de intentarlo. Cada solución viene con la salida real del programa: si la tuya coincide, está bien aunque el código sea distinto. Los `.java` de esta carpeta son los mismos que se muestran abajo, listos para ejecutar: `cd soluciones/paso02` y `javac *.java`.

B2.1 - Adivina la salida

Las seis líneas tal cual, sin cambiarles nada. Lo que se corrige es tu predicción.

▼ Ver la solución (B2_1_Adivina.java)

B2_1_Adivina.java

```
public class B2_1_Adivina {  
    public static void main(String[] args) {  
        System.out.println("" + 1 + 2);  
        System.out.println(1 + 2 + "");  
        System.out.println("Total: " + 10 + 5);  
        System.out.println("Total: " + (10 + 5));  
        System.out.println(10 + 5 + " pesos");  
        System.out.println('a' + 1);  
    }  
}
```

Salida:

```
12  
3  
Total: 105  
Total: 15  
15 pesos  
98
```

Por qué cada una. El `+` se lee **de izquierda a derecha**, y en cuanto uno de los dos lados es texto, el resultado es texto.

Línea	Qué hace Java	Resultado
<code>" " + 1 + 2</code>	texto vacío + 1 da <code>"1"</code> , y <code>"1" + 2</code> da <code>"12"</code>	<code>12</code>
<code>1 + 2 + " "</code>	1 + 2 son números y dan 3; recién ahí aparece el texto	<code>3</code>
<code>"Total: " + 10 + 5</code>	pega el 10, después pega el 5	<code>Total: 105</code>
<code>"Total: " + (10 + 5)</code>	los paréntesis suman primero	<code>Total: 15</code>
<code>10 + 5 + " pesos"</code>	suma primero, porque el texto viene al final	<code>15 pesos</code>
<code>'a' + 1</code>	un <code>char</code> sumado a un número se convierte en su código : la <code>a</code> es 97	<code>98</code>

La última es la que más sorprende. Si quisieras la letra siguiente tendrías que pedir el `char` de vuelta: `(char) ('a' + 1)` imprime `b`.

B2.2 - La boleta

Cuatro `printf` con el mismo molde y un `println` para la línea de guiones.

▼ Ver la solución (B2_2_Boleta.java)

B2_2_Boleta.java

```
public class B2_2_Boleta {
    public static void main(String[] args) {
        System.out.printf("%-14s%4d\n", "Completo", 2500);
        System.out.printf("%-14s%4d\n", "Bebida", 1200);
        System.out.printf("%-14s%4d\n", "Papas fritas", 1800);
        System.out.println("-----");
        System.out.printf("%-14s%4d\n", "TOTAL", 5500);
    }
}
```

Salida:

Completo	2500
Bebida	1200
Papas fritas	1800

TOTAL	5500

Lo que importa. El `-` de `%-14s` es lo que alinea a la **izquierda**; sin el `-`, el texto se pega a la derecha y la boleta queda al revés. El `%4d` no lleva `-` justamente porque los números se leen mejor alineados a la derecha, con las unidades una debajo de la otra.

Si los tres productos te quedaron con `println` y espacios contados a mano, funciona igual... hasta que cambias un nombre. El molde de `printf` es lo que hace que la tabla aguante datos distintos.

B2.3 - La tarjeta

Se puede en **uno solo**: los `\n` hacen los saltos y los espacios se cuentan dentro del texto. La versión de abajo usa tres para que se lea mejor.

▼ Ver la solución (B2_3_Tarjeta.java)

B2_3_Tarjeta.java

```
public class B2_3_Tarjeta {
    public static void main(String[] args) {
        System.out.println("+-----+");
        System.out.println("| DUOC UC           |\n|");
        System.out.println("| Ana Perez         |\n| Seccion 003D");
        System.out.println("+-----+");
    }
}
```

Salida:

```

+-----+
| DUOC UC          |
|                  |
| Ana Perez        |
| Seccion 003D     |
+-----+

```

Lo que importa. Que veas que `\n` **no es** lo mismo que un `println`: es un carácter más dentro del texto. Un solo `println` puede imprimir seis líneas.

Con `\t` no queda bien: la tabulación salta a la columna 8, 16, 24..., así que el borde derecho se descuadra apenas un nombre es más largo. Para dibujar cuadros se usan espacios; el `\t` sirve para columnas, no para bordes.

B2.4 - Las comillas

Tres `println` con caracteres de escape.

▼ Ver la solución (B2_4_Cita.java)

B2_4_Cita.java

```

public class B2_4_Cita {
    public static void main(String[] args) {
        System.out.println("El profesor dijo: \"Java obliga, Python avisa\"");
        System.out.println("La ruta es C:\\duoc\\java");
        System.out.println("Fin\\tdel\\tejercicio");
    }
}

```

Salida:

```

El profesor dijo: "Java obliga, Python avisa"
La ruta es C:\duoc\java
Fin      del      ejercicio

```

Lo que importa. La barra invertida es el carácter de escape, así que para imprimir **una** hay que escribir **dos**. Es el error clásico al escribir rutas de Windows: `"C:\duoc"` no compila, porque Java lee `\d` y no sabe qué escape es ese.

Si te dio `illegal escape character`, era eso.

B2.5 - Desafío: la tabla alineada

Las dos versiones en el mismo programa, para ver la diferencia. La primera separa con espacios escritos a mano; la segunda usa anchos fijos.

▼ Ver la solución (B2_5_Tabla.java)

B2_5_Tabla.java

```
public class B2_5_Tabla {
    public static void main(String[] args) {
        // Version 1: espacios a mano. Se descuadra con el numero mas largo.
        System.out.println("Producto      Precio    IVA");
        System.out.println("Completo      2500.0   475.0");
        System.out.println("Bebida      1200.0   228.0");
        System.out.println("Torta      12500.0 2375.0");

        System.out.println();

        // Version 2: printf con anchos fijos. Aguanta cualquier dato.
        System.out.printf("%-16s%8s%8s%n", "Producto", "Precio", "IVA");
        System.out.printf("%-16s%8.1f%8.1f%n", "Completo", 2500.0, 475.0);
        System.out.printf("%-16s%8.1f%8.1f%n", "Bebida", 1200.0, 228.0);
        System.out.printf("%-16s%8.1f%8.1f%n", "Torta", 12500.0, 2375.0);
    }
}
```

Salida:

Producto	Precio	IVA
Completo	2500.0	475.0
Bebida	1200.0	228.0
Torta	12500.0	2375.0

Producto	Precio	IVA
Completo	2500.0	475.0
Bebida	1200.0	228.0
Torta	12500.0	2375.0

Lo que importa. Mira la fila de la torta en las dos versiones: en la primera, el precio tiene un dígito más y empuja toda la fila; en la segunda, `%8.1f` reserva **siempre** 8 espacios, así que el número crece hacia la izquierda y la columna no se mueve.

Regla práctica: si la tabla la arma un `for` con datos que no controlas (el paso 7), los espacios a mano **siempre** se descuadran. Por eso existe `printf`.

Volver a la [hoja del paso 2](#) o al [paso 2 de la guía](#).